UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

ECE408 - CS483 - CSE408 Spring 2026
Applied Parallel Programming

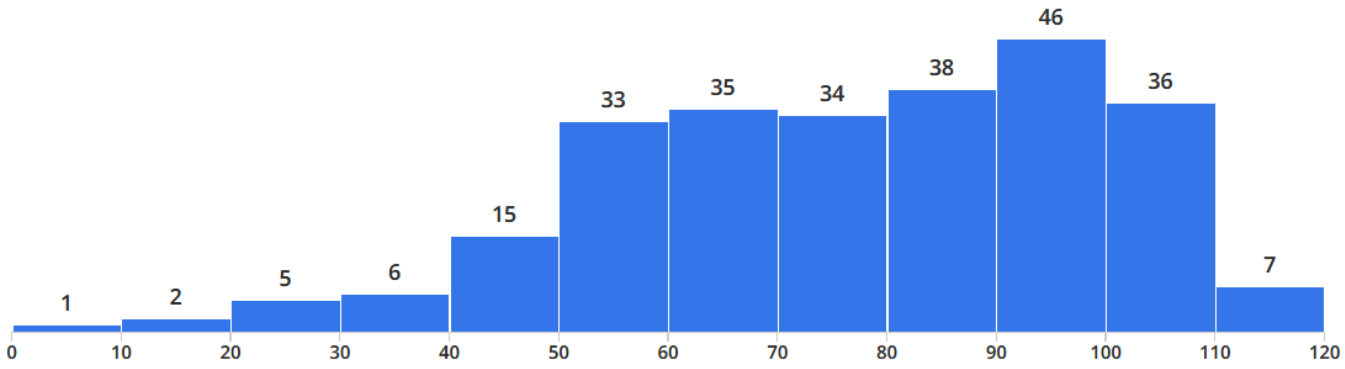
Lecture 15: Parallel Computation Patterns – Reduction Trees

Volodymyr Kindratenko

3/12/2026

Course Reminders

- Labs
  - Lab 5 due this Friday
  - Lab 6 is also out, it is due in 2 weeks
  - **Please check your grades for labs 1-4 and make sure they are graded correctly**
- Project
  - Project milestone 2 will be released soon
- Midterm 1
  - Graded, check you grades and submit regrade requests via GradeScope by this Friday midnight.



Minimum	Median	Maximum	Mean	Std Dev
4.0	79.0	118.0	76.95	22.14

Today's Objectives

- To learn the basic concept of reductions, one of the most widely used parallel computation patterns
- To learn simple strategies for parallelization of reductions
- To understand the performance issues involved with performing reductions on GPUs
- To learn about warp-level primitives and how they can be used to share data with other threads in the same warp
- To learn about warp-level programming with Cooperative Groups

Important Enough to Use in Theory

"… scan operations, also known as prefix computations,

can execute in no more time than …
parallel memory references …

greatly simplify the description of many [parallel]
algorithms, and are
significantly easier to implement than
memory references." —Guy Blelloch, 1989*

*G. Blelloch, "Scans as Primitive Parallel Operations,"
IEEE Transactions on Computers, 38(11):1526-1538, 1989.
The idea behind scans for computation goes back another 30+ years.

Trying to Bridge Theory and Practice

- A generic parallel algorithm,
  - in which parallel threads access memory arbitrarily,
  - is likely to produce an extremely slow access pattern.
- Scans
  - can be implemented quickly in hardware, and
  - form a useful alternative to arbitrary memory accesses.

(His hope was to enable theory
without knowledge of microarchitecture.)

Reduction

- A **reduction** operation reduces a set of input values to one value
  - e.g., sum, product, min, max

- Reduction operations are:
  - Associative
  - Commutative
  - Have a well-define identity value

- We will use **sum** as an example

Sequential Reduction is O(N)

Given binary operator ▫ and an identity value $I^{▫}$

- $I^{▫}$ = 0 for sum
- $I^{▫}$ = 1 for product
- $I^{▫}$ = largest possible value for min
- $I^{▫}$ = smallest possible value for max

```
result ← I▫
for each value X in input
    result ← result ▫ X
```

Sequential Reduction

- Sequential reduction for sum:

```
sum = 0.0f;
for(i = 0; i < N; ++i) {
    sum += input[i];
}
```

- In general:

```
acc = IDENTITY;
for(i = 0; i < N; ++i) {
    acc = f(acc, input[i]);
}
```

Parallel Reduction with Atomics

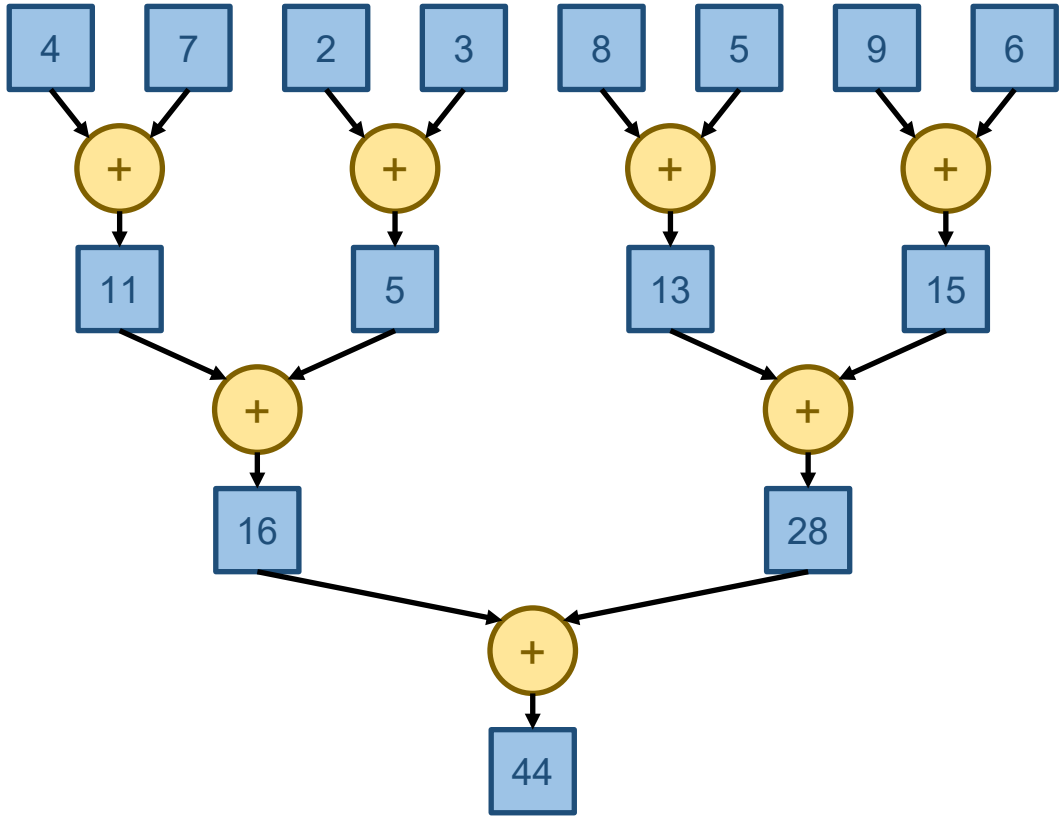
- Parallel reduction for sum using atomics:

```
unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
if(i < N) {
    atomicAdd(sum, input[i]);  // sum is a pointer to global memory
}
```

- Poor performance:
  - All additions are serialized by the hardware

Parallel Reduction in log(N) Steps

- Approach: Every thread adds two elements in each step
- Takes log(N) steps and half the threads drop out every step
- Pattern is called a **reduction tree**



Algorithm is Work Efficient

- For ***N* input values**, the **number of operations** is

$$\frac{1}{2}N + \frac{1}{4}N + \frac{1}{8}N + \cdots + \frac{1}{N}N = \left(1 - \frac{1}{N}\right)N = \boldsymbol{N - 1}.$$

- The parallel algorithm shown is **work-efficient**:
  - requires the **same amount of work as a sequential algorithm**
  - (constant overheads, but nothing dependent on ***N***).

Fast if Enough Resources are Available

- For ***N* input values**, the **number of steps** is **log (N)**.
- With enough execution resources,
  - **N=1,000,000 takes 20 steps!**
  - Sounds great!
- **How much parallelism** do we need?
  - **On average, (N-1)/log(N)**.
    50,000 in our example.
  - But **peak is N/2**!
    500,000 in our example.

Diminishing Parallelism is Common

- **In** our **parallel reduction**,
  - the **number of operations**
  - **halves in every step**.
- This kind of **narrowing parallelism is common**
  - from combinational logic circuits
  - to basic blocks
  - to high-performance applications.

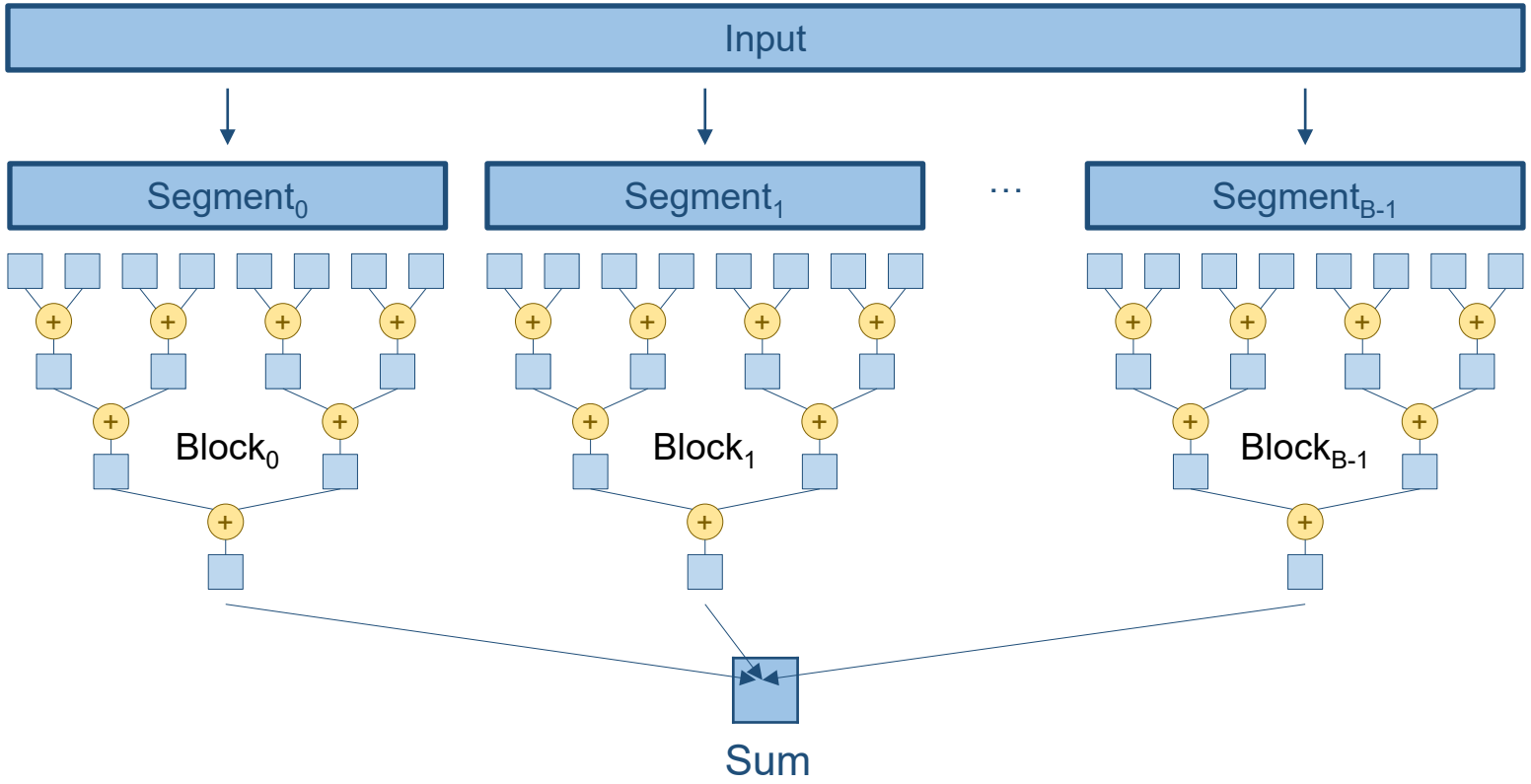
CUDA kernels allow only a fixed number of threads.

Segmented Reduction

- Threads must synchronize between steps
  - Cannot synchronize across threads in different blocks

- Solution: **segmented reduction**
  - Every thread block reduces a segment of the input and produces a partial sum
  - The partial sum is atomically added to the final sum

Segmented Reduction



Parallel Strategy for CUDA

- Let's start simple: ***N*** values in device global memory.
- Each **thread block** of ***M*** threads
  - uses global memory,
  - to **reduce chunk of 2M** values to one value
  - (2M << N to produce enough thread blocks).
- Blocks operate **within global memory**
  - Store all intermediate results, and
  - **write one value back** to global memory at the end.

CUDA Reduction Algorithm

- **Read** block of **2M values** from global memory.

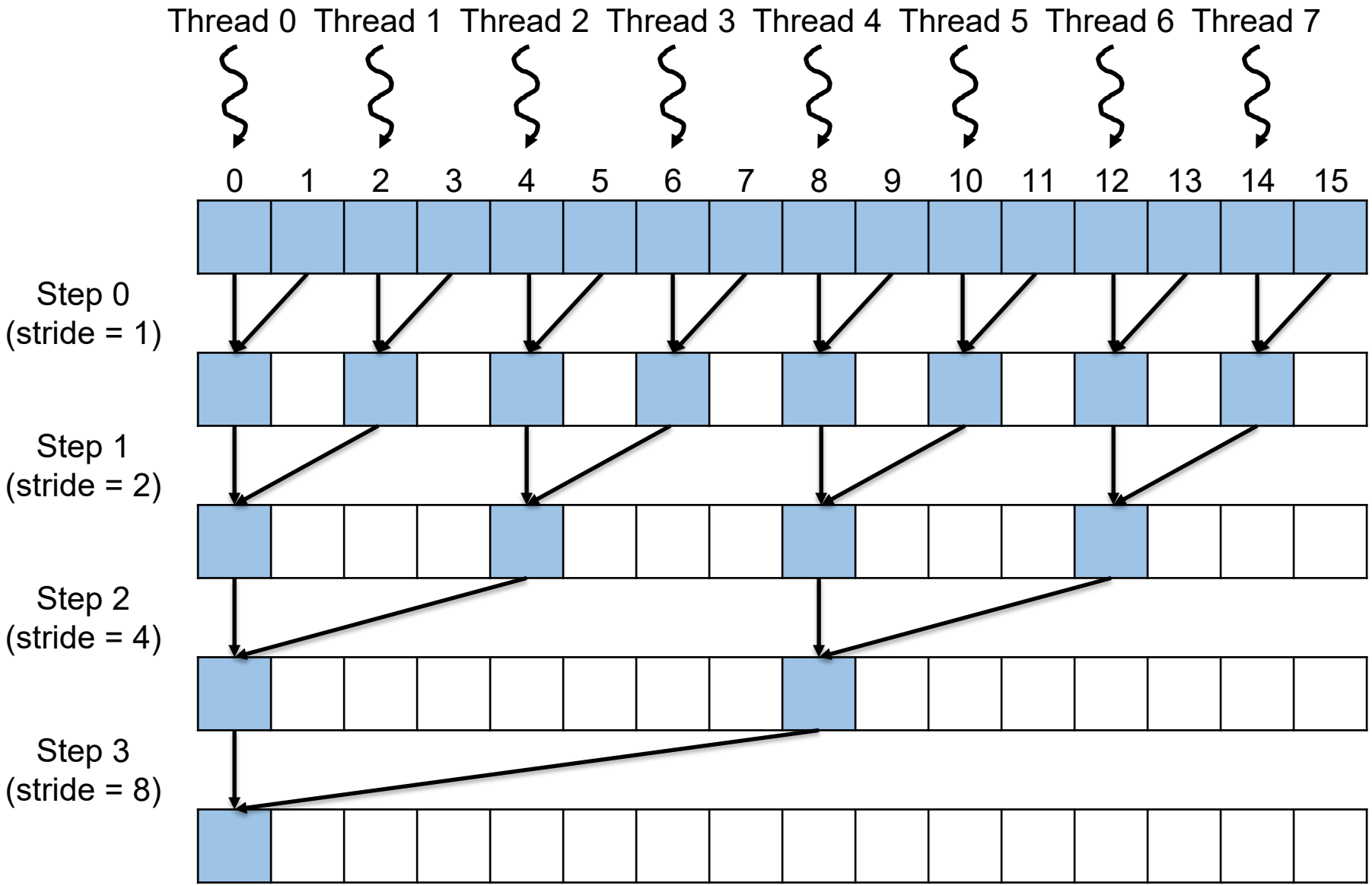
- For each of **log(M)** steps,
  - **combine two values** per thread in each step,
  - **write result** to global memory, and
  - **halve** the number of **active threads**.

- **Write final result** back to global memory.

A Simple Mapping of Data to Threads

- Each **thread**
  - **begins with** two **adjacent locations** (**stride of 1**),
  - **even index (first)** and an odd index (second).
  - Thread 0 gets 0 and 1, Thread 1 gets 2 and 3, …
  - Write **result** back **to** the **even index**.
- **After each step,**
  - **half of** active **threads** are **done**.
  - **Double the stride**.
- At the end, **result is at index 0**.

Reduction Tree (Per Block Segment)

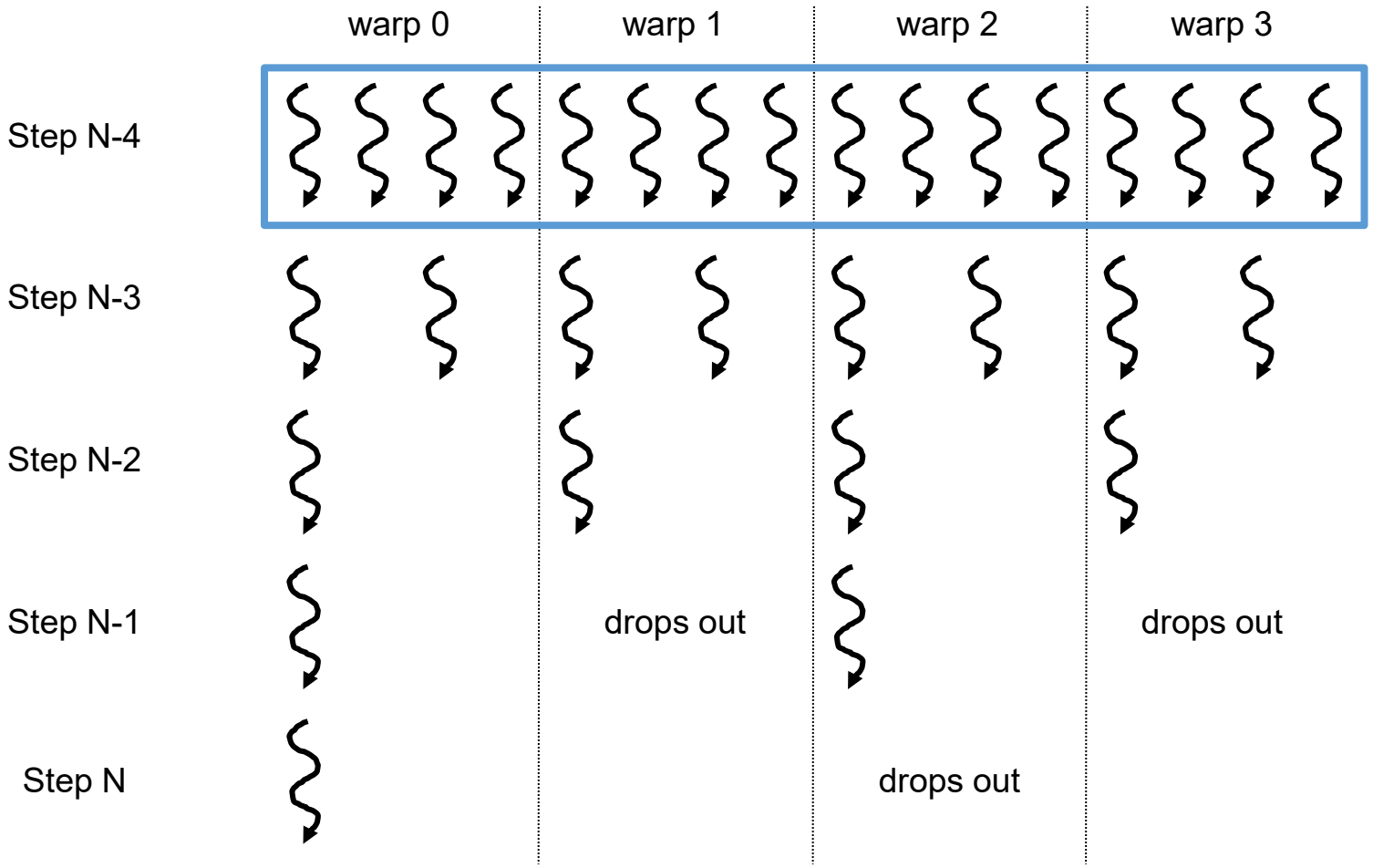


Code for Reduction Tree Per Block Segment

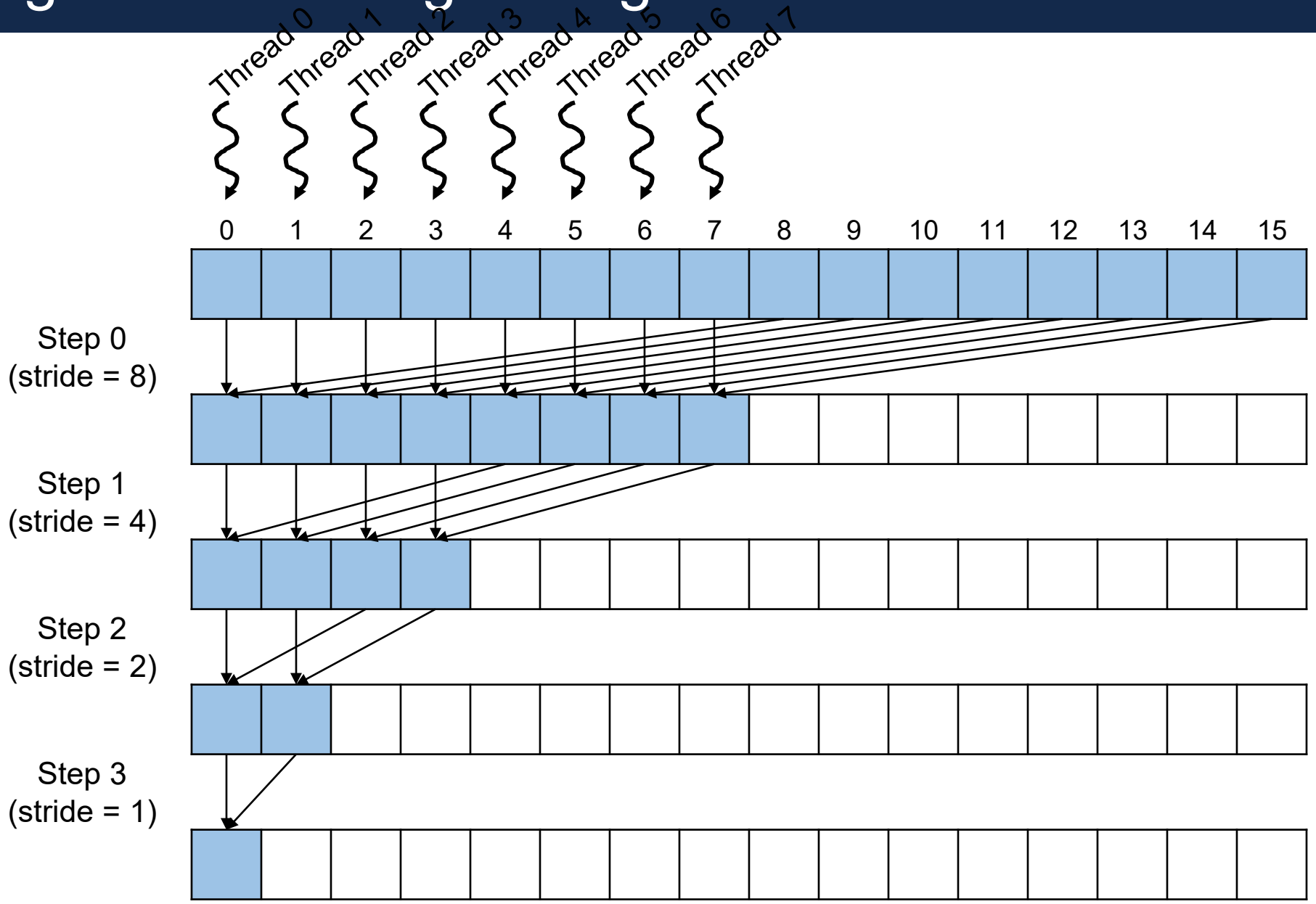
```
unsigned int segment = 2*blockDim.x*blockIdx.x;
unsigned int i = segment + 2*threadIdx.x;
for(unsigned int stride = 1; stride <= BLOCK_DIM; stride *= 2) {
    if(threadIdx.x % stride == 0) {
        input[i] += input[i + stride];
    }
    __syncthreads();
}
```

- Problems:
  - Accesses to `input` are not coalesced
  - Control divergence

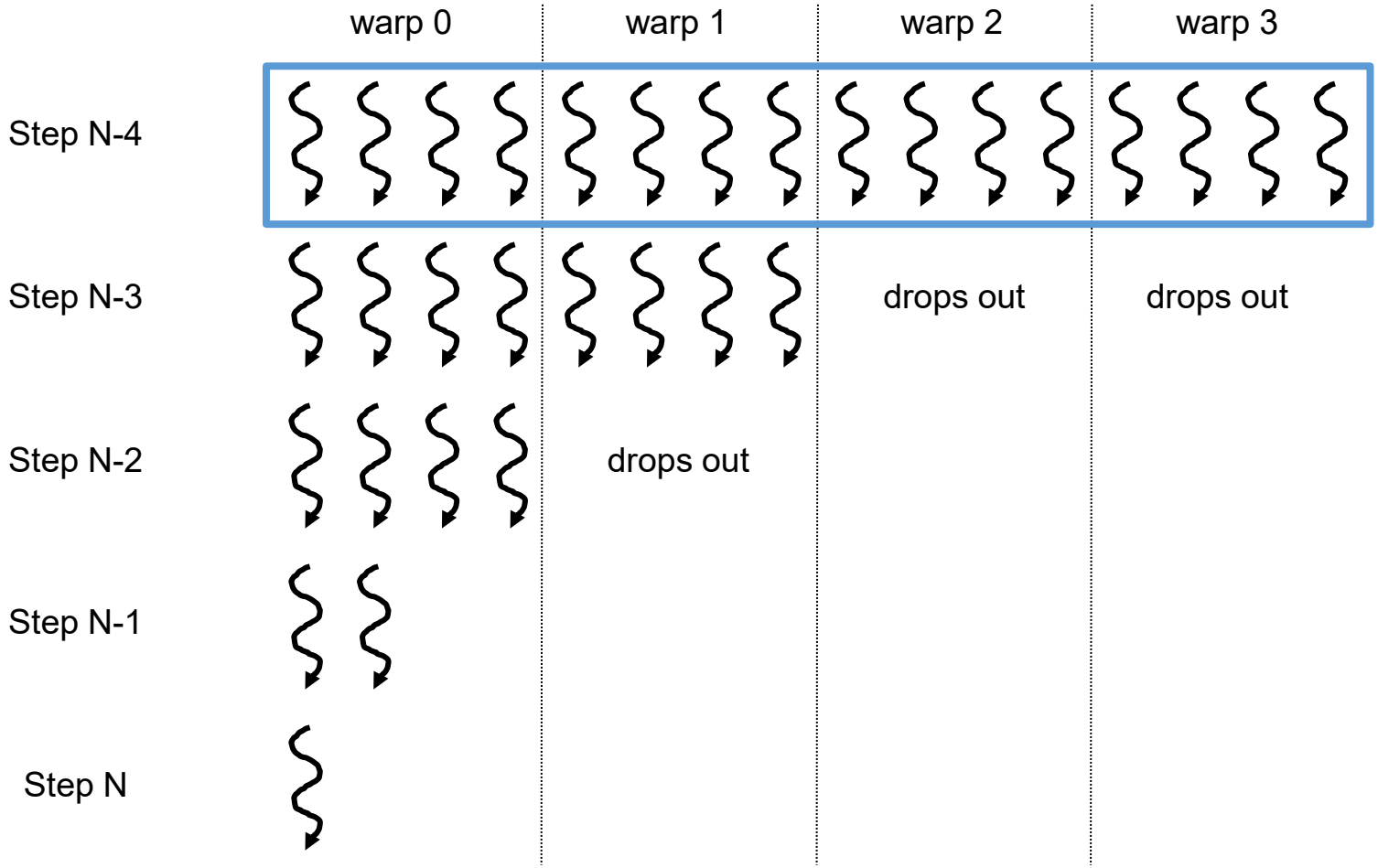
Control Divergence Problem



Coalescing and Reducing Divergence



Control Divergence Reduced



Code for Reduction with Coalescing & Reducing Divergence

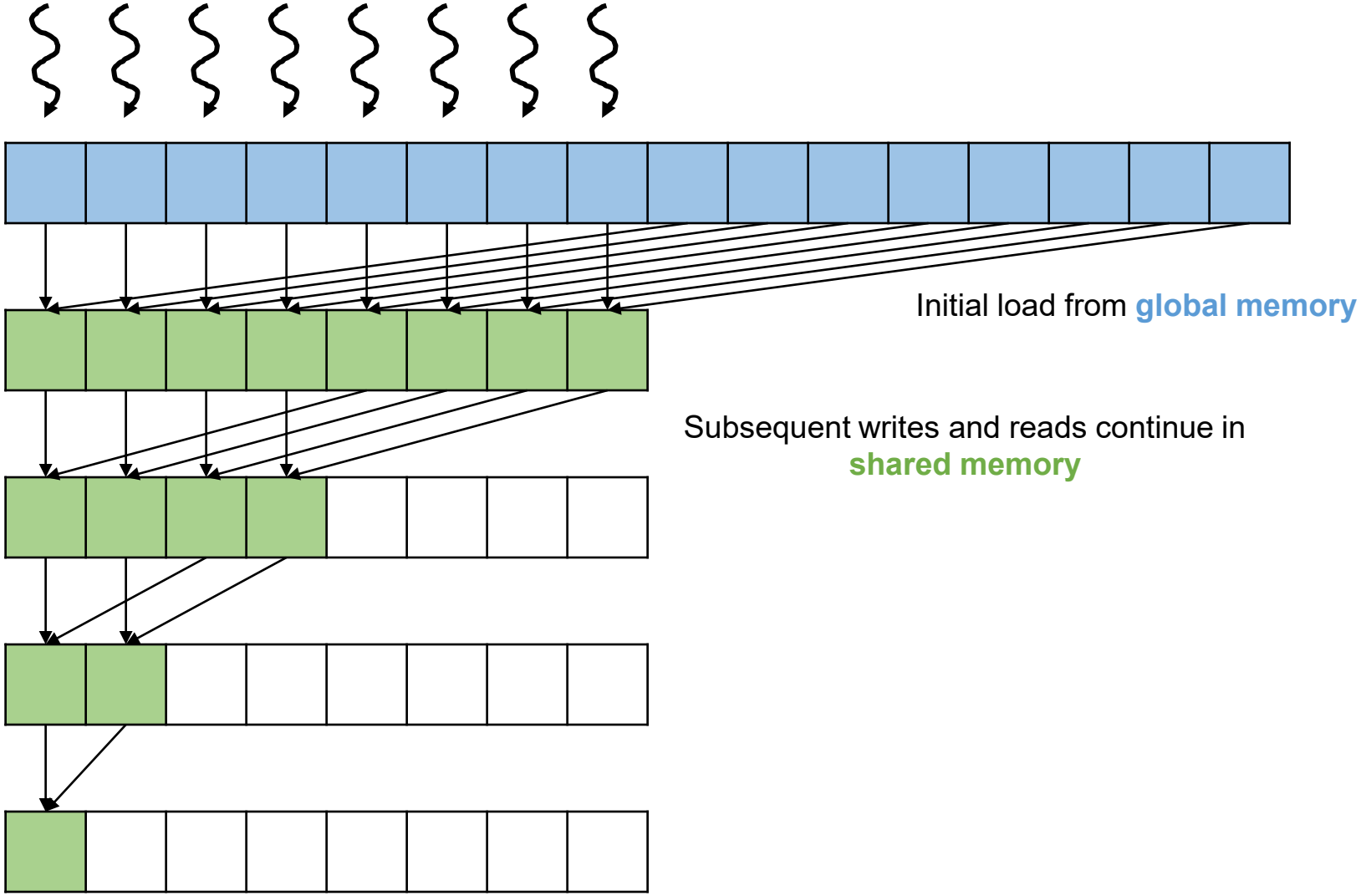
```
unsigned int segment = 2*blockDim.x*blockIdx.x;
unsigned int i = segment + threadIdx.x;
for(unsigned int stride = BLOCK_DIM; stride > 0; stride /= 2) {
    if(threadIdx.x < stride) {
        input[i] += input[i + stride];
    }
    __syncthreads();
}
```

Data Reuse

- While specific data values are not reused, the same memory locations are repeatedly read and written

- Optimization: load input to shared memory first and perform reduction tree on shared memory
- Also avoids modifying the input if needed in the future

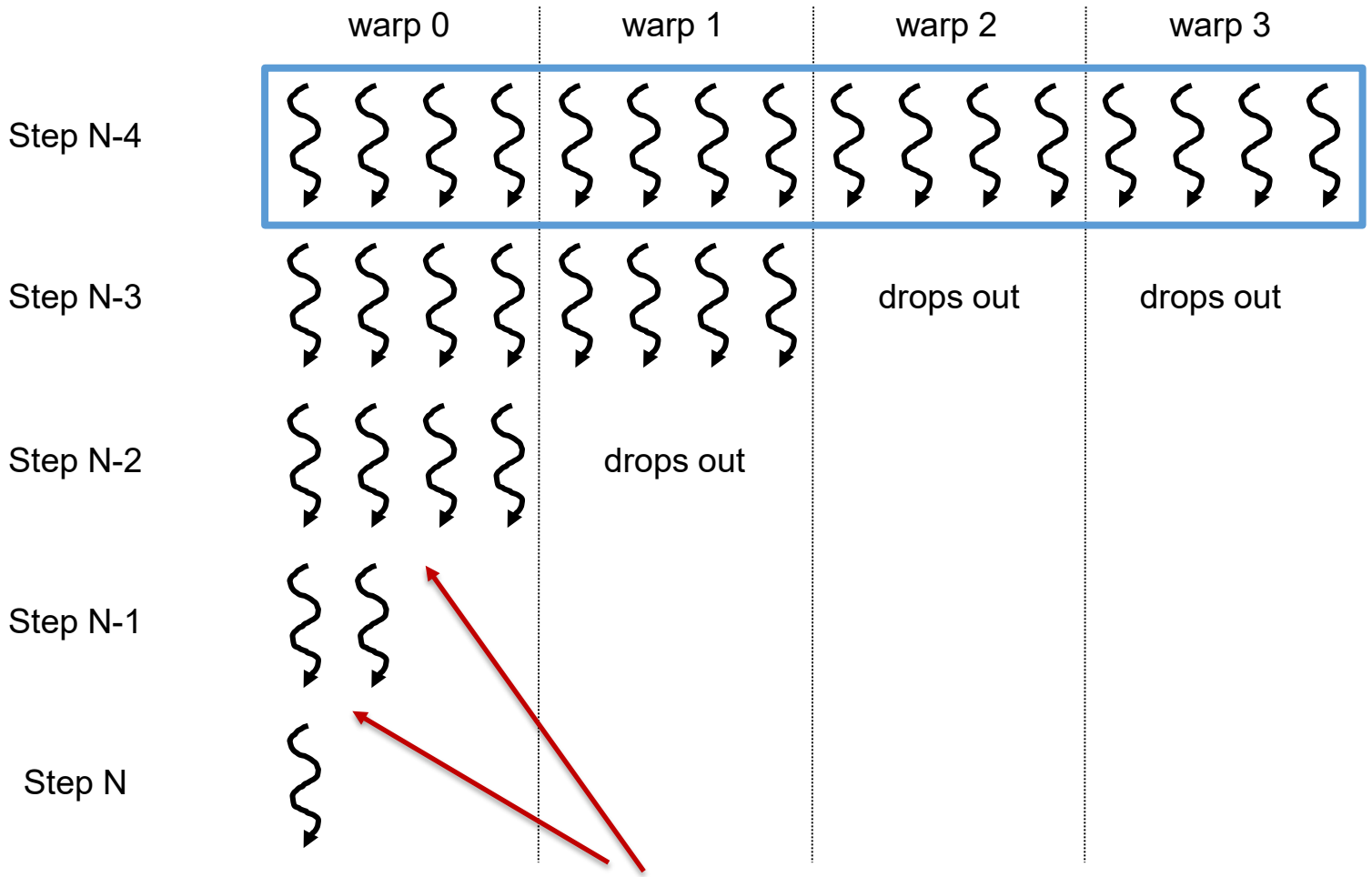
Using Shared Memory



Code for Reduction with Shared Memory

```
unsigned int segment = 2*blockDim.x*blockIdx.x;
unsigned int i = segment + threadIdx.x;

// Load data to shared memory
__shared__ float input_s[BLOCK_DIM];
input_s[threadIdx.x] = input[i] + input[i + BLOCK_DIM];
__syncthreads();

// Reduction tree in shared memory
for(unsigned int stride = BLOCK_DIM/2; stride > 0; stride /= 2) {
    if(threadIdx.x < stride) {
        input_s[threadIdx.x] += input_s[threadIdx.x + stride];
    }
    __syncthreads();
}
```

Reducing Synchronization

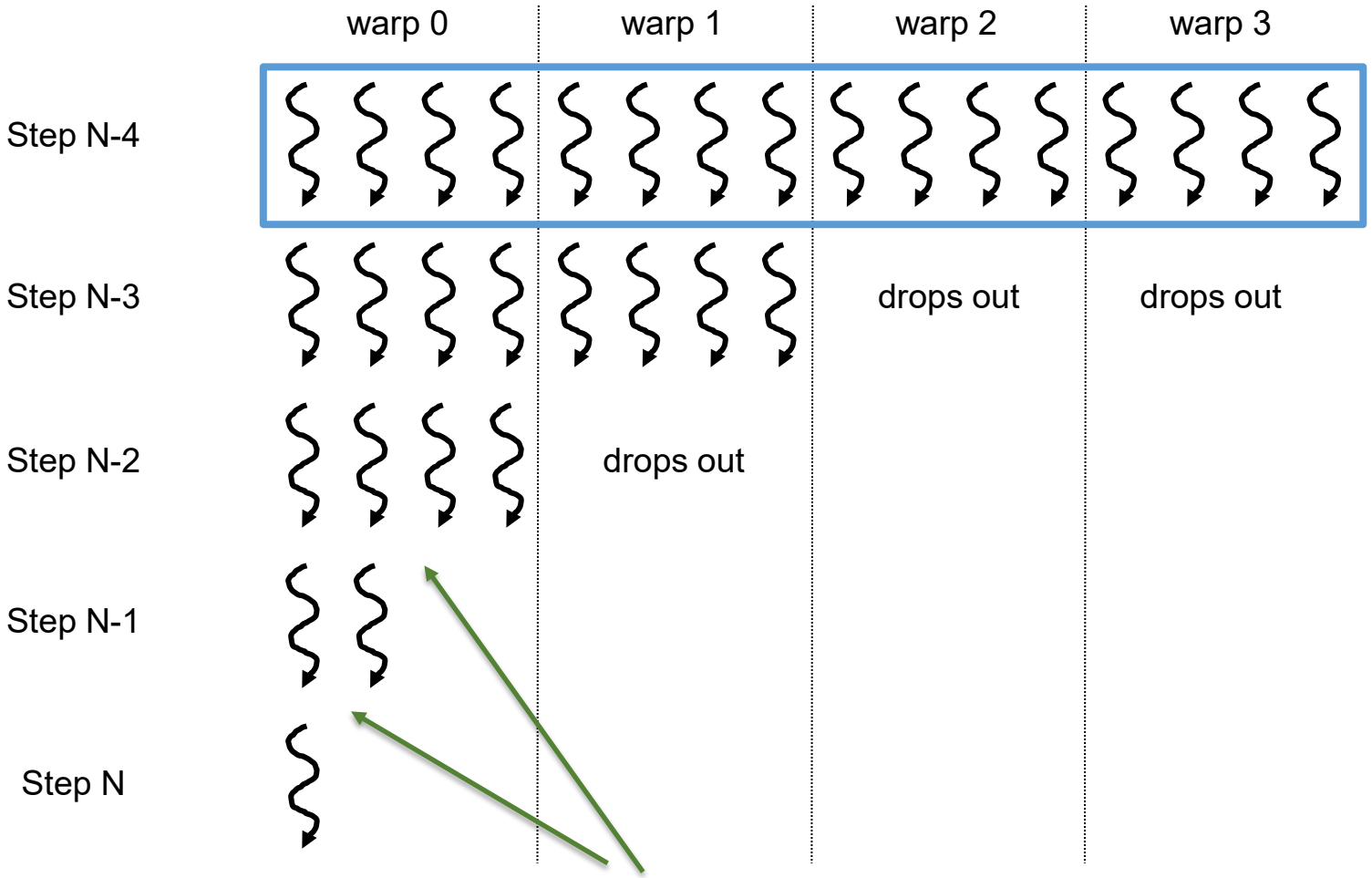


When one warp remains, using shared memory and barrier synchronization across the entire thread block is unnecessary

Reducing Synchronization with Warp-level Primitives

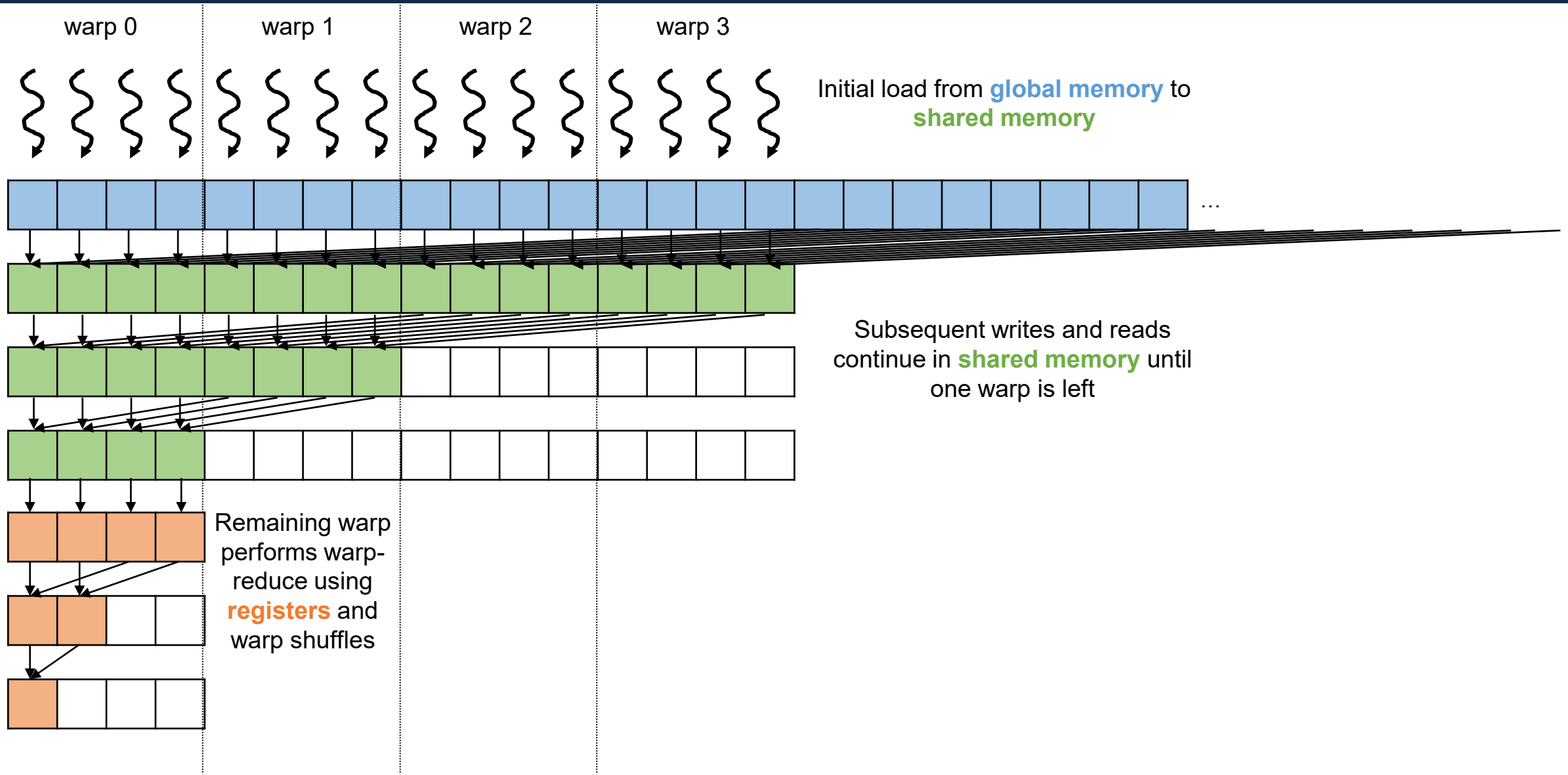
- During the last few iterations, only one warp is active
  - We can take advantage of the special relationship between threads in the same warp to synchronize between them quickly
- Built-in **warp shuffle functions** enable threads to share data with other threads in the same warp
  - Faster than using shared memory and `__syncthreads()` to share across threads in the same block
- Variants:
  - `__shfl_sync()`
    - Direct copy from indexed lane
  - `__shfl_up_sync()`
    - Copy from a lane with lower ID relative to caller
  - `__shfl_down_sync()`
    - Copy from a lane with higher ID relative to caller
  - `__shfl_xor_sync()`
    - Copy from a lane based on bitwise XOR of own lane ID

Reduction with Warp Shuffle



When one warp remains, use warp shuffle instructions to synchronize within the warp and share data from registers

Reduction with Two-Stage Warp Reductions



Code for Reduction with Warp Shuffle

```
__device__ __inline__ float warpReduce(float val) {
    float partialSum = val;
    for(unsigned int stride = WARP_SIZE/2; stride > 0; stride /= 2) {
        partialSum += __shfl_down_sync(0xffffffff, partialSum, stride);
    }
    return partialSum;
}

    ...

    // Reduction tree in shared memory
    for(unsigned int stride = BLOCK_DIM/2; stride > WARP_SIZE; stride /= 2) {
        if(threadIdx.x < stride) {
            input_s[threadIdx.x] += input_s[threadIdx.x + stride];
        }
        __syncthreads();
    }

    if(threadIdx.x < WARP_SIZE) {

        // Reduction tree with shuffle instructions
        float partialSum = input_s[threadIdx.x]
                         + input_s[threadIdx.x + WARP_SIZE];
        partialSum = warpReduce(partialSum);

    }
```

Warp-level Programming with Cooperative Groups

- The **cooperative groups** API is a convenient way of synchronizing across a group of threads
  - Available at multiple levels:
    - Entire grid
    - Thread block cluster
    - Thread block
    - Block tile (including multi-warp, single-warp, and sub-warp)
- To use cooperative groups:

```
#include <cooperative_groups.h>
using namespace cooperative_groups;
```

- To get a handle on the warp:

```
thread_block_tile<32> warp =
tiled_partition<32>(this_thread_block());
```

Warp-level Programming with Cooperative Groups

- To perform a warp shuffle:

```
__shfl_down_sync(0xffffffff, partialSum, stride)
```

↓

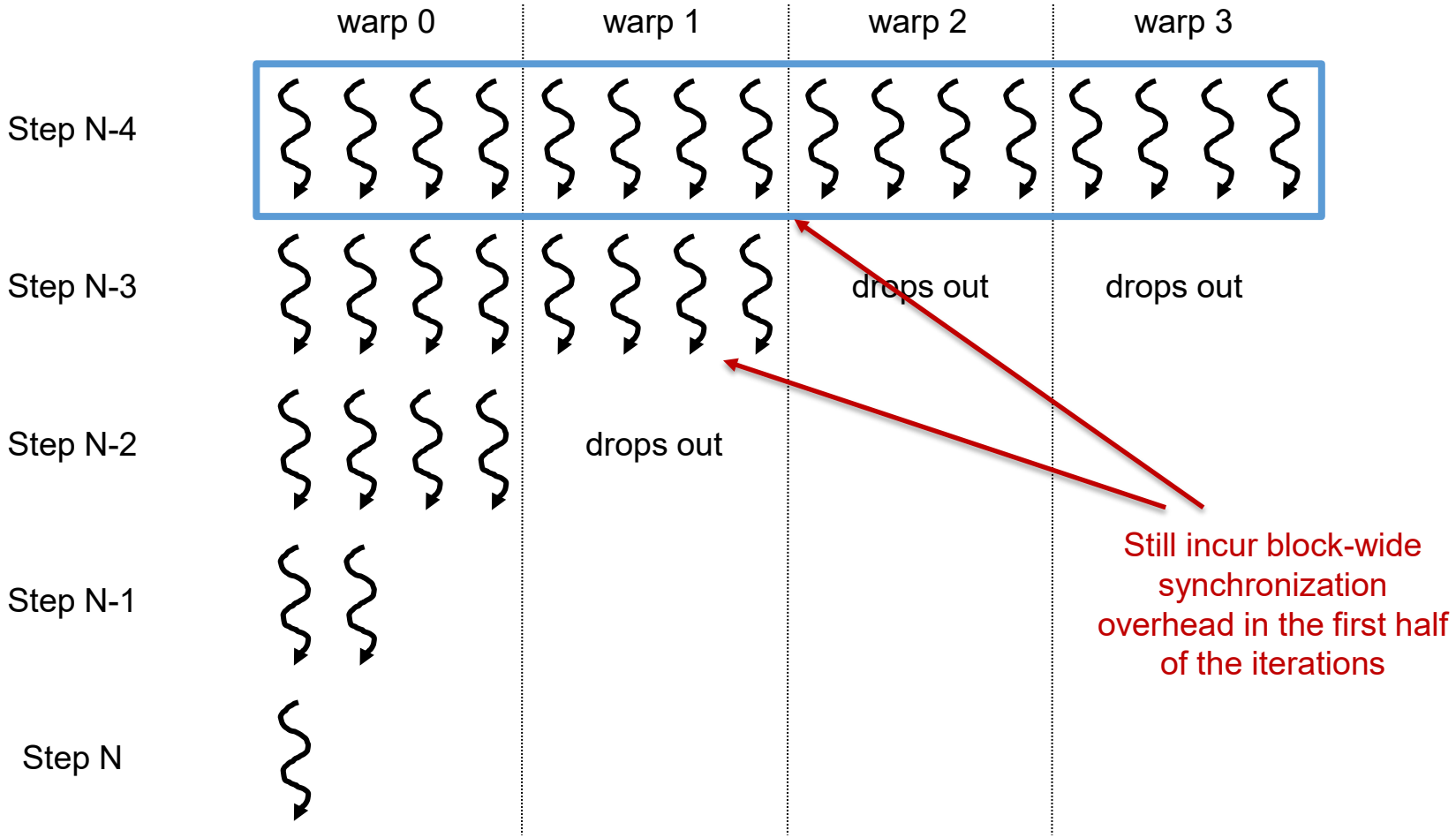
```
warp.shfl_down(partialSum, stride)
```

- Other features of cooperative groups covered later

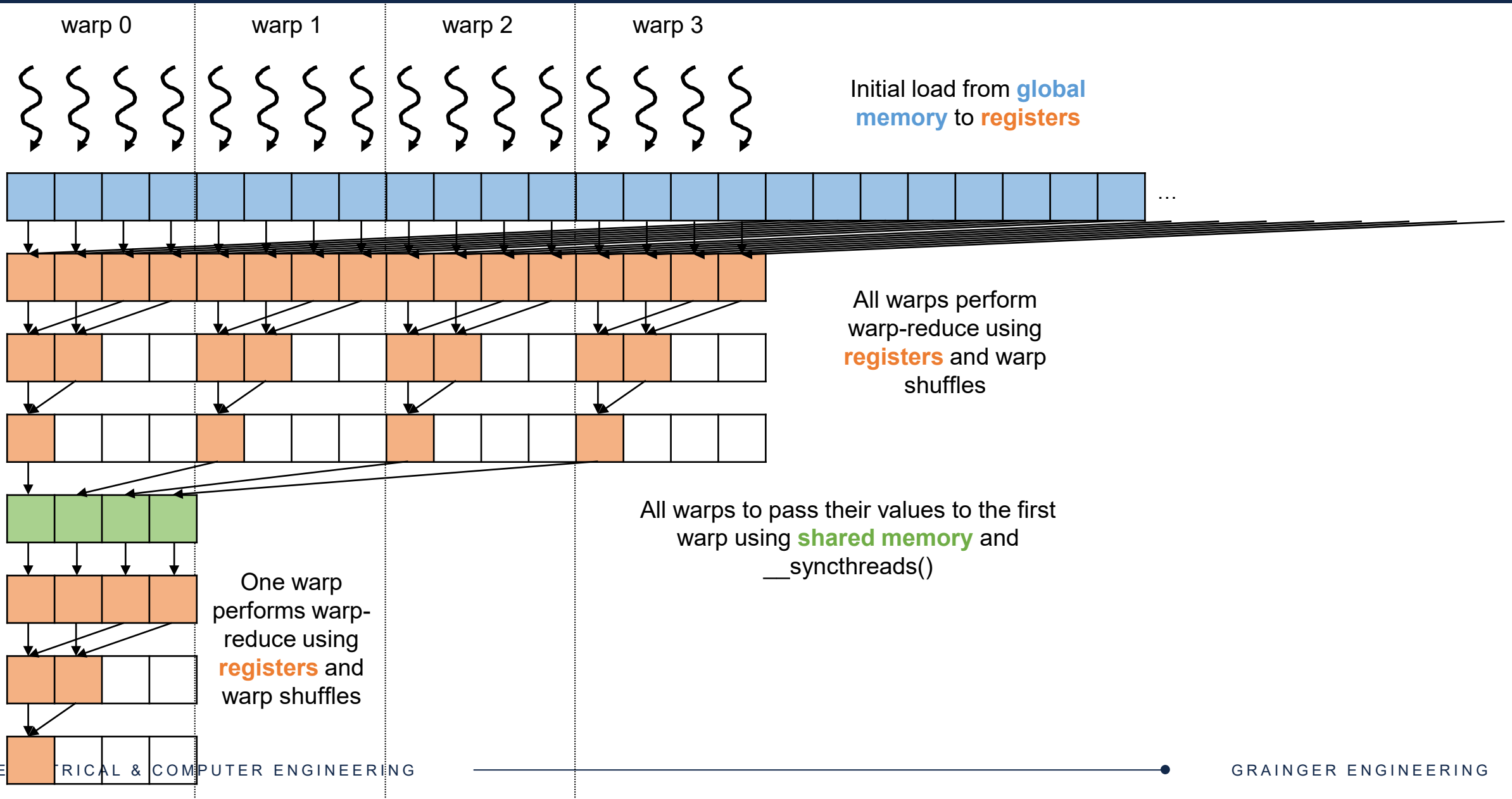
Reduction with Warp Shuffle Code using Cooperative Groups

```
#include <cooperative_groups.h>
using namespace cooperative_groups;

__device__ __inline__ float warpReduce(float val) {
    thread_block_tile<32> warp = tiled_partition<32>(this_thread_block());
    float partialSum = val;
    for(unsigned int stride = warp.size()/2; stride > 0; stride /= 2) {
        partialSum += warp.shfl_down(partialSum, stride);
    }
    return partialSum;
}
```

Synchronization in Reduction with Warp Shuffle



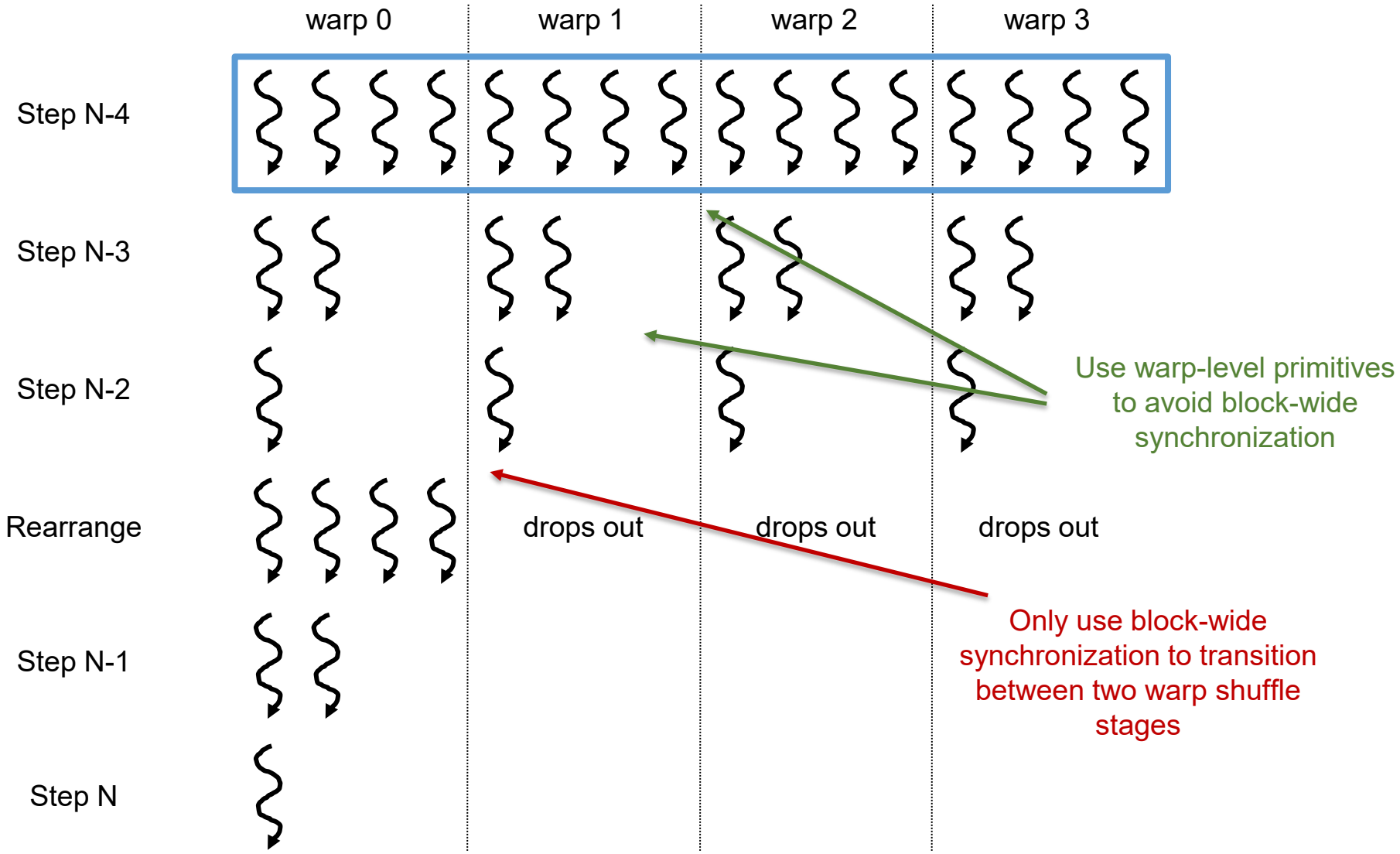
Reduction with Two-Stage Warp Reductions



Code for Reduction with Two-Stage Warp Reductions

```
unsigned int segment = 2*blockDim.x*blockIdx.x;
unsigned int i = segment + threadIdx.x;

// Load data to registers
float partialSum = input[i] + input[i + BLOCK_DIM];

// Reduction tree with shuffle instructions
partialSum = warpReduce(partialSum);

// Combine partial sums in shared memory
__shared__ float partialSums_s[BLOCK_DIM/WARP_SIZE];
if(threadIdx.x % WARP_SIZE == 0) {
    partialSums_s[threadIdx.x/WARP_SIZE] = partialSum;
}
__syncthreads();

if(threadIdx.x < WARP_SIZE) {

    // Reduction tree with shuffle instructions
    float partialSum = partialSums_s[threadIdx.x];
    partialSum = warpReduce(partialSum);
}
```

Divergence in Reduction with Two-Stage Warp Reductions



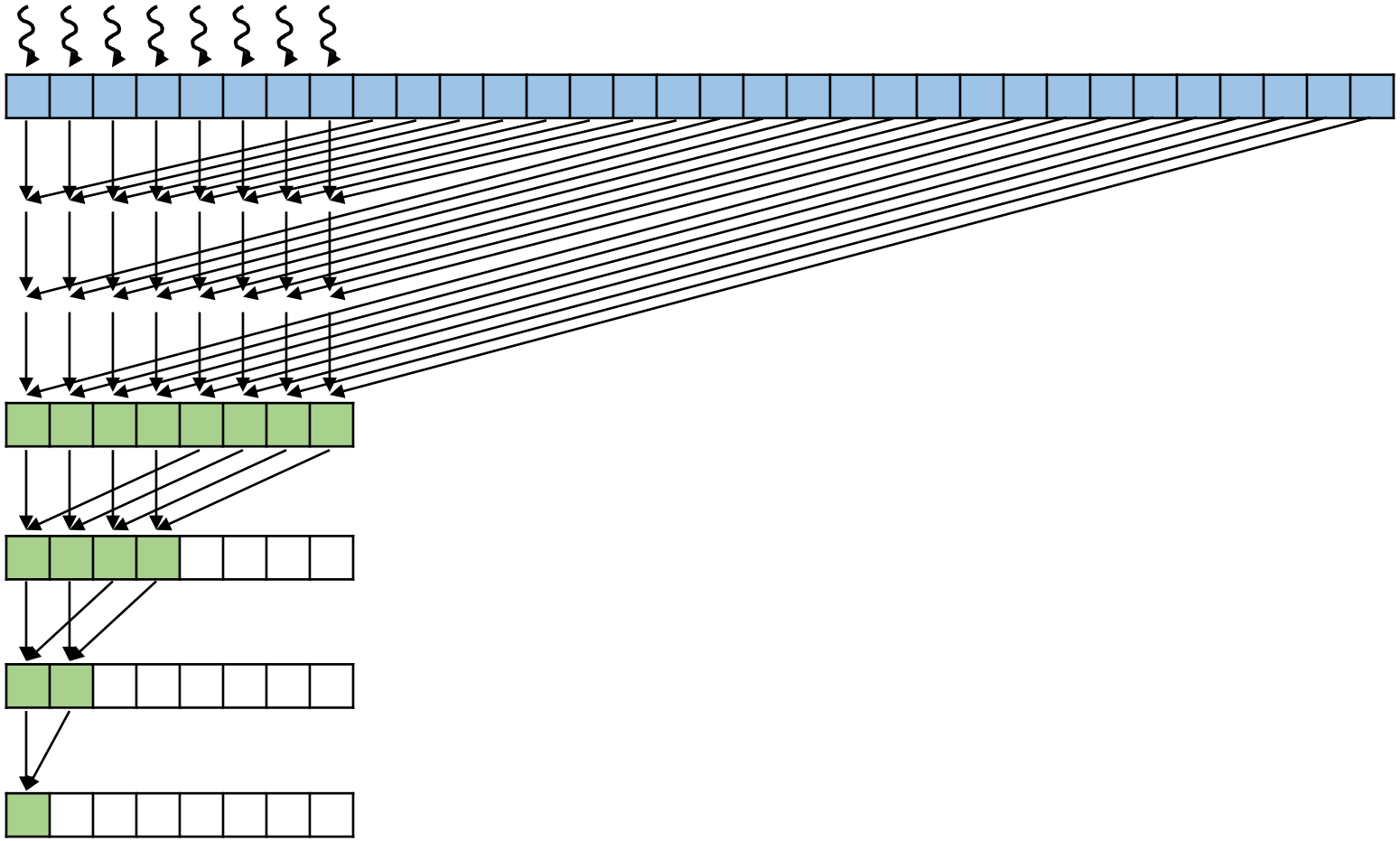
Trade-off: More control divergence, less barrier synchronization and shared memory accesses

Thread Coarsening

- Cost of parallelization:
  - Synchronization every step
  - Control divergence in the final steps

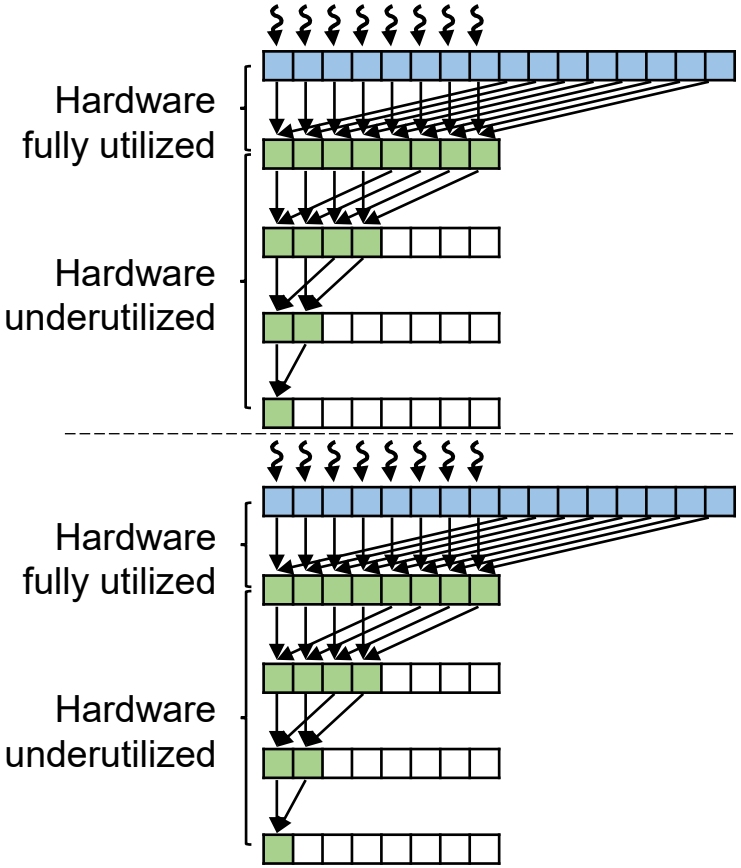
- Better to coarsen threads if there are many more blocks than resources available

Applying Thread Coarsening

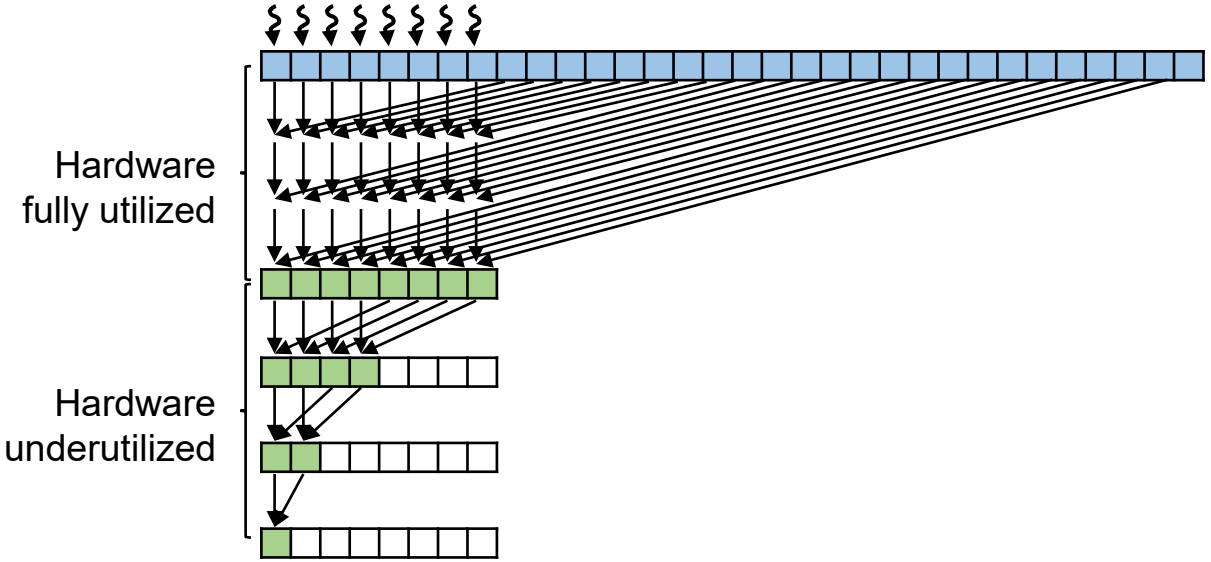


Before vs. After Coarsening

Execution of two non-coarsened thread blocks serialized by the hardware



Execution of one coarsened thread block doing the work of two original non-coarsened thread blocks



Code for Reduction with Thread Coarsening

```
unsigned int segment = COARSE_FACTOR*2*blockDim.x*blockIdx.x;
unsigned int i = segment + threadIdx.x;

// Load data to registers
float partialSum = 0.0f;
for(unsigned int tile = 0; tile < COARSE_FACTOR*2; ++tile) {
    partialSum += input[i + tile*BLOCK_DIM];
}

// Reduction tree with shuffle instructions
partialSum = warpReduce(partialSum);

// Combine partial sums in shared memory
__shared__ float partialSums_s[BLOCK_DIM/WARP_SIZE];
if(threadIdx.x % WARP_SIZE == 0) {
    partialSums_s[threadIdx.x/WARP_SIZE] = partialSum;
}
__syncthreads();

if(threadIdx.x < WARP_SIZE) {

    // Reduction tree with shuffle instructions
    float partialSum = partialSums_s[threadIdx.x];
    partialSum = warpReduce(partialSum);
}
```

Coarsening Benefits

- Let *N* be the number of elements per original block
  - i.e., *N* = 2*blockDim.x
- If blocks are all executed in parallel:
  - *log(N)* steps, *log(N)* synchronizations
- If blocks serialized by the hardware by a factor of C:
  - *C*log(N)* steps, *C*log(N)* synchronizations
- If blocks are coarsened by a factor of C:
  - *2*(C – 1) + log(N)* steps, *log(N)* synchronizations

Any More Questions?

Things to Read:

Textbook chapter 10
CUDA PG: 10.22. Warp Shuffle Functions
CUDA PG: Chapter 11. Cooperative Groups

Things to Do:

Lab 5
Lab 6